

Importer Motocykli

(poleasingowe.pl)

Kompletna dokumentacja systemu

Wtyczka WordPress + scraper Python + osobna baza MySQL

Dokumentacja dla użytkownika biznesowego oraz administratora technicznego.

Produkt	Podstrona „Nasze motory” z aukcjami motocykli z poleasingowe.pl
Wersja wtyczki	0.7.2
Autor	Krzysztof Leszczyński
Licencja	GPL-2.0-or-later
Wymagania	WordPress 6.0+ / PHP 7.4+ • MySQL 5.7+/MariaDB 10.2+ • Python 3 (VPS)

Spis treści

Część I. Dokumentacja dla osoby nietechnicznej	3
1. Czym jest system w skrócie	3
2. Co widzi osoba odwiedzająca stronę	3
3. Skąd pochodzą dane (proces biznesowy)	3
4. Aktualność danych i cykl życia oferty	4
5. Zarządzanie systemem przez właściciela strony	4
6. Najczęstsze pytania (część nietechniczna)	4
7. Czego system nie robi (ograniczenia)	4
Część II. Dokumentacja dla osoby technicznej	6
8. Architektura systemu	6
9. Zawartość repozytorium (branch plugin-2)	6
10. Integracja ze źródłem (poleasingowe.pl)	7
11. Baza danych — schemat	7
12. Scraper — pipeline i moduły	8
13. Wtyczka WordPress — moduły	10
14. Interfejsy i „endpointy”	10
15. Instalacja i wdrożenie — krok po kroku (A-Z)	11
16. Konfiguracja — zmienne środowiskowe scrapera	12
17. Bezpieczeństwo (podsumowanie)	13
18. Utrzymanie i eksploatacja	13
19. Wydajność	14
Część III. Załączniki i informacje wymagające potwierdzenia	15
20. Wymaga potwierdzenia przez zespół projektowy	15
21. Słownik pojęć	15
22. Odnośniki (dokumentacja w repozytorium)	16

CZĘŚĆ I

Dokumentacja dla osoby nietechnicznej

Do czego służy system, co widzi klient i jak nim zarządzać

1. Czym jest system w skrócie

System automatycznie pobiera oferty aukcji **motocykli** z serwisu **poleasingowe.pl** i prezentuje je na stronie internetowej klienta na dedykowanej podstronie **„Nasze motory”**. Podstrona tworzy się sama po włączeniu wtyczki i wyglądem dopasowuje się do dowolnego motywu (kolory i czcionki dziedziczy ze strony klienta).

System składa się z dwóch współpracujących części, opisanych szczegółowo w części technicznej:

- **Wtyczka WordPress** — pokazuje motocykle na stronie klienta (lista, filtry, szczegóły).
- **Automat pobierający dane (scraper)** — działa na serwerze VPS i co pewien czas pobiera aktualne aukcje ze źródła, zapisując je do **osobnej bazy danych**.

Dzięki rozdzieleniu tych części strona klienta jest szybka i stabilna: nawet gdy źródło chwilowo nie odpowiada, strona nadal pokazuje ostatnio pobrane oferty.

2. Co widzi osoba odwiedzająca stronę

Na podstronie **„Nasze motory”** odwiedzający zobaczy:

- **Listę (siatkę) motocykli** — kafelki ze zdjęciem, marką i modelem, ceną oraz podstawowymi parametrami: rok, przebieg, pojemność, paliwo.
- **Filtry** nad listą: marka, paliwo, rok, cena do (PLN). Po wybraniu filtrów lista sama się zawęża.
- **Podział na strony (paginacja)** — gdy ofert jest dużo, dzielą się na kolejne strony.
- **Widok pojedynczego motocykla** — po kliknięciu kafelka: galeria zdjęć oraz tabela szczegółów (VIN, rok produkcji, przebieg, pojemność, moc, paliwo, skrzynia, napęd, kolor, lokalizacja, numer aukcji) i przycisk **„Zobacz aukcję na poleasingowe.pl”**.

Komunikaty, które może zobaczyć odwiedzający:

- **„Oferta motocykli będzie dostępna wkrótce.”** — system nie ma jeszcze połączenia z bazą danych (zadanie dla administratora — patrz część techniczna).
- **„Brak motocykli spełniających kryteria.”** — żadna oferta nie pasuje do wybranych filtrów.
- **„Nie znaleziono tego motocykla.”** — link prowadzi do oferty, której już nie ma w bazie (np. aukcja się zakończyła).

3. Skąd pochodzą dane (proces biznesowy)

1. Automat łączy się ze stroną **poleasingowe.pl**, konkretnie z kategorią **motocykle**.
2. Pobiera listę wszystkich aktualnych aukcji, a następnie wchodzi w każdą z nich po szczegóły i zdjęcia.
3. Sprawdza poprawność danych (m.in. czy jest cena, marka, sensowny rok) i odrzuca wpisy błędne.
4. Zapisuje uporządkowane dane do **osobnej bazy danych** (nie miesza się z bazą WordPressa).
5. Wtyczka na stronie klienta **czyta** te dane i wyświetla je odwiedzającym.

Zdjęcia nie są kopiowane na serwer klienta — są pokazywane bezpośrednio ze źródła (tzw. **hotlink**). Oznacza to mniej miejsca na dysku, ale zależność od dostępności zdjęć w źródle.

4. Aktualność danych i cykl życia oferty

Dane odświeżają się **automatycznie co pewien czas** (harmonogram na serwerze). To nie jest podgląd „na żywo” — między odświeżeniami może upłynąć kilka godzin.

Każda oferta przechodzi przez następujące stany:

- **Aktywna** — oferta jest dostępna w źródle i pokazywana na stronie.
- **Zakończona** — oferta zniknęła ze źródła (aukcja się skończyła). System **nie kasuje** jej od razu — oznacza jako zakończoną i przestaje pokazywać na liście aktywnych.
- **Wznowienie (relist)** — gdy ten sam motocykl (rozpoznany po numerze VIN) pojawia się ponownie jako nowa aukcja, system rozpoznaje, że to powtórka.

Dobrze wiedzieć

Ceny bywają podawane jako „netto”. Jeżeli w danych brakuje ceny, na stronie pojawia się napis „Cena do ustalenia”.

5. Zarządzanie systemem przez właściciela strony

Do codziennej obsługi nie jest potrzebna wiedza techniczna. W panelu WordPress dostępny jest ekran **Ustawienia → Motocykle**, na którym można:

- sprawdzić **status połączenia z bazą** (czy system widzi dane i ile jest motocykli),
- przejść do podstrony „**Nasze motory**”,
- **wyczyścić pamięć podręczną (cache)** listy — przydatne, gdy chcemy natychmiast zobaczyć najnowsze dane bez czekania.

Podstronę można edytować jak każdą inną stronę WordPress. Za wyświetlanie motocykli odpowiada krótki znacznik `[motocykle]` wstawiony w treść strony — nie należy go usuwać.

6. Najczęstsze pytania (część nietechniczna)

Widzę „Oferta będzie dostępna wkrótce” — co robić?

Oznacza to brak konfiguracji połączenia z bazą danych. To jednorazowe zadanie dla administratora technicznego (patrz część II, rozdział „Instalacja i wdrożenie”).

Lista jest pusta.

Możliwe przyczyny: wybrane filtry nie pasują do żadnej oferty, albo automat pobierający dane nie wykonał jeszcze importu. Warto wyczyścić filtry i sprawdzić status w Ustawienia → Motocykle.

Zdjęcia się nie wyświetlają.

Zdjęcia pochodzą bezpośrednio ze źródła (hotlink). Jeżeli źródło je usunęło lub zablokowało, mogą się nie pokazać, mimo że reszta danych jest poprawna.

Dane wyglądają na nieaktualne.

System odświeża dane cyklicznie, nie w czasie rzeczywistym. Można przyspieszyć pokazanie najnowszych danych, czyszcząc cache listy w panelu.

7. Czego system nie robi (ograniczenia)

- Nie pośredniczy w licytacji ani zakupie — kieruje do oryginalnej aukcji na poleasingowe.pl.
- Obsługuje **wyłącznie kategorię motocykle** (nie samochody ani inne pojazdy).
- Dane są odświeżane cyklicznie — to nie jest podgląd na żywo.

- Zdjęcia są pokazywane ze źródła (hotlink), więc ich dostępność zależy od poleasingowe.pl.

CZĘŚĆ II

Dokumentacja dla osoby technicznej

Architektura, moduły, instalacja, konfiguracja, wdrożenie i utrzymanie

8. Architektura systemu

System składa się z trzech elementów połączonych osobną bazą danych MySQL:

- **Scraper (Python)** — uruchamiany cyklicznie na serwerze VPS; pobiera i przetwarza aukcje, jest **właścicielem danych** (zapisuje do bazy).
- **Baza MySQL `polea\_\*`** — osobna baza (tabele `polea_motocykle`, `polea_zdjecia`); pełni rolę warstwy pośredniej między scraperem a stroną.
- **Wtyczka WordPress** — czyta bazę **tylko do odczytu** i renderuje podstronę „Nasze motory”.

Przepływ danych: **poleasingowe.pl** → **scraper (pipeline działów)** → **MySQL `polea\_\*`** → **wtyczka** → **podstrona „Nasze motory”**. Projekt jest zorganizowany w „działy” z rolami agent (wykonanie) i krytyk (weryfikacja) — szczegółowy schemat architektury dostarczany jest jako osobny załącznik graficzny.

Kluczowa decyzja projektowa

Wtyczka NIE tworzy własnych wpisów/typów treści (CPT) w WordPressie. Aukcje są czasowe, a ich właścicielem jest scraper, dlatego front działu bezpośrednio na osobnej bazie (rozwiązanie „DB-driven”). Wtyczka łączy się przez `mysqli` z obsługą błędów, a nie przez `wpdb` — dzięki temu awaria bazy nie wywoła `wp_die` i nie wyłączy strony klienta.

9. Zawartość repozytorium (branch plugin-2)

Poniżej wszystkie istotne pliki wraz z rolą:

Plik / katalog	Rola
<code>db/schema.sql</code>	Schemat osobnej bazy MySQL (tabele <code>polea_motocykle</code> , <code>polea_zdjecia</code>).
<code>scraper/config.py</code>	Konfiguracja i zmienne środowiskowe scrapera (URL, limity, dane bazy).
<code>scraper/dzial7_zgodnosc.py</code>	Bramka zgodności: <code>robots.txt</code> , <code>rate-limit</code> , <code>anti-SSRF</code> , limity, <code>retry</code> .
<code>scraper/dzial1a_lista.py</code>	Dział 1A — crawl listy aukcji kategorii motocykle.
<code>scraper/dzial1b_szczegoly.py</code>	Dział 1B — parsowanie strony szczegółów + zdjęcia.
<code>scraper/dzial2_normalizacja.py</code>	Dział 2 — normalizacja pól, walidacja VIN, jednostki.
<code>scraper/dzial5_audyt.py</code>	Dział 5 — reguły audytu (twarde/miękkie) przed zapisem.
<code>scraper/dzial3_deduplikacja.py</code>	Dział 3 — deduplikacja <code>lot_id</code> + wykrywanie relistów po VIN.
<code>scraper/dzial4_synchronizacja.py</code>	Dział 4 — <code>upsert</code> do bazy + <code>reconcile</code> (zamykanie nieobecnych).
<code>scraper/main.py</code>	Orkiestrator pipeline'u + blokada pojedynczej instancji (<code>flock</code>).
<code>scraper/requirements.txt</code>	Zależności: <code>requests</code> , <code>PyMySQL</code> (rdzeń działu na <code>stdlib</code>).
<code>scraper/tests/test_scraper.py</code>	Testy jednostkowe (16), w tym testy <code>anti-SSRF</code> ; bez sieci i bazy.
<code>wp-plugin/motocykle-poleasingowe/</code>	Katalog wtyczki WordPress (do wgrania do <code>wp-content/plugins</code>).
<code>.../motocykle-poleasingowe.php</code>	Główny plik wtyczki: nagłówek, stałe, <code>hooks</code> .

Plik / katalog	Rola
.../includes/class-db.php	Klasa Polea_DB — połączenie i zapytania (odczyt) do bazy.
.../includes/security.php	Sanityzacja wejścia, allowlisty, nonce/uprawnienia, cache.
.../includes/shortcode.php	Shortcode [motocykle]: lista, szczegóły, filtry, paginacja.
.../includes/activation.php	Tworzenie podstrony „Nasze motory” i wpięcie w menu.
.../includes/admin.php	Ekran Ustawienia → Motocykle (status, cache, instrukcja).
.../uninstall.php	Sprzątanie przy usunięciu wtyczki (nie dotyczy bazy polea_*).
.../assets/front.css	Style frontu dopasowane do motywu klienta.
docs/ARCHITEKTURA.md, docs/dzialy/*	Dokumentacja architektury i poszczególnych działów.
docs/SECURITY-AUDIT.md	Raport audytu bezpieczeństwa (2 iteracje utwardzania).

10. Integracja ze źródłem (poleasingowe.pl)

Źródło serwuje **gotowy HTML po stronie serwera (bez JavaScript)**, dlatego scraper używa biblioteki requests i wyrażeń regularnych — **bez** przeglądarki/Playwright. Wykorzystywane adresy (endpoints źródła):

Zasób	Adres (wzorzec)
Lista aukcji (kategoria motocykle)	https://poleasingowe.pl/pl/auctions/list/pub/all/ecr_motorcycles?page=N
Szczegóły pojedynczej aukcji	<a href="https://poleasingowe.pl/pl/auctions/details/<slug>/<lot_id>">https://poleasingowe.pl/pl/auctions/details/<slug>/<lot_id>
Zdjęcie (hotlink)	<a href="https://poleasingowe.pl/images/sgallery_<UUID>_75.png">https://poleasingowe.pl/images/sgallery_<UUID>_75.png
robots.txt	https://poleasingowe.pl/robots.txt (respektowany przez scraper)

- `lot_id` z adresu szczegółów to **stały klucz** danej aukcji (np. 9pm53mj9).
- Reklamy partnerów prowadzące do innych domen (np. `aukcje.pkoleasing.pl`) są **pomijane** — wzorzec linku wymaga ścieżki `/pl/auctions/details/` .
- Paginacja: parametr `?page=N` , przechodzona aż do wyczerpania nowych ofert (bezpiecznik `POLEA_MAX_PAGES`).

11. Baza danych — schemat

Silnik **InnoDB** , kodowanie **utf8mb4** . Tabela główna `polea_motocykle` (jeden wiersz = jedna aukcja):

Kolumna	Typ	Znaczenie
<code>lot_id</code>	VARCHAR(32) PK	Stały identyfikator lotu z adresu URL.
<code>numer_aukcji</code>	VARCHAR(64)	Numer aukcji, np. 3110/BZ/AU/2026.
<code>slug</code>	VARCHAR(255)	Człon adresu URL oferty.
<code>url</code>	VARCHAR(512)	Pełny adres strony szczegółów.
<code>marka / model / typ</code>	VARCHAR	Dane pojazdu.
<code>rok_produkcji</code>	SMALLINT UNSIGNED	Rok produkcji.
<code>data_pierwszej_rej</code>	DATE	Data pierwszej rejestracji.
<code>vin</code>	VARCHAR(20)	Numer VIN (walidowany, patrz Dział 2).

Kolumna	Typ	Znaczenie
nr_rej	VARCHAR(32)	Numer rejestracyjny.
naped / skrzynia	VARCHAR(64)	Rodzaj napędu / skrzynia biegów.
moc_km	SMALLINT UNSIGNED	Moc w KM.
pojemnosc_ccm	INT UNSIGNED	Pojemność w ccm.
paliwo / kolor	VARCHAR	Rodzaj paliwa / kolor.
przebieg_km	INT UNSIGNED	Przebieg w km.
ilosc_kluczykow	TINYINT UNSIGNED	Liczba kluczyków.
forma_sprzedazy	VARCHAR(64)	Np. faktura VAT.
cena_pln	DECIMAL(12,2)	Aktualna cena.
cena_netto	TINYINT(1)	1 = cena netto.
najnizsza_cena_30d	DECIMAL(12,2)	Najniższa cena z 30 dni (patrz Załączniki).
tryb_licytacji	VARCHAR(64)	Tryb licytacji (patrz Załączniki).
lokalizacja	VARCHAR(255)	Lokalizacja pojazdu.
termin_zakonczenia	DATETIME	Data/godzina końca aukcji.
status	VARCHAR(32)	aktywna zakończona usunięta (domyślnie aktywna).
liczba_ofert	INT UNSIGNED	Liczba ofert (patrz Załączniki).
uwagi	TEXT	Uwagi (patrz Załączniki).
raw_hash	CHAR(32)	MD5 rekordu — wykrywanie zmian (nowy/zmieniony/bez zmian).
first_seen / last_seen / updated_at	TIMESTAMP	Znaczniki czasu cyklu życia rekordu.

Indeksy: klucz główny `lot_id` oraz indeksy `marka`, `status`, `termin_zakonczenia`, `vin`, `last_seen`.

Tabela `polea_zdjecia` (wiele zdjęć na jeden lot):

Kolumna	Typ	Znaczenie
id	BIGINT UNSIGNED PK	Auto-increment.
lot_id	VARCHAR(32)	Powiązanie z aukcją (FK → <code>polea_motocykle</code> , ON DELETE CASCADE).
image_key	VARCHAR(64)	UUID zdjęcia (z nazwy pliku).
url	VARCHAR(512)	Pełny adres zdjęcia (hotlink).
sort_order	SMALLINT UNSIGNED	Kolejność wyświetlania.

Unikalność (`lot_id`, `image_key`) zapewnia idempotentny zapis zdjęć (ponowny import nie duplikuje).

12. Scraper — pipeline i moduły

Kolejność przetwarzania (orkiestратор `main.py`): **1A** → **1B** → **2** → **5** → **3** → **4**, a wszystkie żądania sieciowe przechodzą przez bramkę **Działu 7**. Rdzeń przetwarzania działa na biblioteczce standardowej; `requests` / `PyMySQL` są importowane leniwie (dzięki temu testy działają bez instalacji).

Dział 7 — Zgodność / bramka sieci (dzial7_zgodnosc.py)

- Respektuje **robots.txt** (pobierany z twardym timeoutem i limitem 512 KB).
- **Rate-limit** : odstęp POLEA_DELAY (domyślnie 2 s) + losowy POLEA_JITTER .
- **Anty-SSRF** : allowlista hostów (tylko poleasingowe.pl i www.poleasingowe.pl), blokada adresów prywatnych/lokalnych/metadanych, **ręczna walidacja każdego przekierowania** (allow_redirects=False).
- **Limit rozmiaru odpowiedzi** POLEA_MAX_BYTES (8 MB) czytany strumieniowo + **budżet czasu** POLEA_MAX_TOTAL (ochrona przed „slow-loris” i bombą dekompresyjną).
- Obsługa kodów: 403 → blokada, 429/503 → backoff, sieć → ponawianie do POLEA_RETRIES .
- Ignoruje proxy/.netrc ze środowiska (trust_env=False) — obrona anty-SSRF na współdzielonym hoście.

Dział 1A — Pobieranie listy (dzial1a_lista.py)

Przechodzi kolejne strony listy i wyłuskuje „stuby” ofert (lot_id , slug , url) wzorcem /pl/auctions/details/<slug>/<lot_id> . Kończy, gdy strona nie przynosi nowych lotów.

Dział 1B — Pobieranie szczegółów (dzial1b_szczegoly.py)

Parsuje pary etykieta/wartość (auction-data-item), og:description (cena), tekst statusu, lokalizację oraz listę zdjęć (sgallery_<UUID>_75.png). Funkcja _clean usuwa znaczniki HTML, znaki sterujące i przycina długość pól (obrona w głąb wobec niezaufanego HTML).

Dział 2 — Normalizacja (dzial2_normalizacja.py)

- Mapuje polskie etykiety źródła na kolumny bazy i konwertuje jednostki (liczby, daty).
- **Waliduje VIN** zgodnie z ISO 3779 (17 znaków, cyfra kontrolna) — miękka flaga vin_valid .
- Nakłada zdroworozsądkowe **górne limity** (moc, pojemność, przebieg, cena) — anty-absurd/overflow.
- Wyznacza cenę z og:description , cena_netto , termin_zakonczenia i status .

Dział 5 — Audyt (dzial5_audyt.py)

Reguły **twarde** (blokują publikację rekordu): brak lot_id , cena ≤ 0, brak marki, rok poza zakresem 1950...rok+1, nieznan status. Reguły **miękkie** (tylko flaga): niepoprawny VIN, brak zdjęć, brak terminu.

Dział 3 — Deduplikacja (dzial3_deduplikacja.py)

Usuwa powtórzenia po lot_id . Wykrywa **relisty** : ten sam **pełny, poprawny VIN** pod nowym lot_id (nigdy nie łączy po pustym/niepoprawnym VIN); najnowszy wpis oznacza jako wiodący, pozostałe wskazują na niego polem relist_of .

Dział 4 — Synchronizacja / zapis (dzial4_synchronizacja.py)

- Liczy raw_hash (MD5) do wykrywania zmian rekordu.
- **Upsert** ofert i zdjęć (INSERT ... ON DUPLICATE KEY UPDATE) — parametryzowane zapytania.
- **Reconcile** : aktywne oferty nieobecne w bieżącym imporcie zmienia na zakończona .
- Transakcja z commit / rollback ; połączenie z timeoutami, wyłączonym LOCAL INFILE i opcjonalnym TLS (POLEA_DB_SSL_CA).

Orkiestrator (main.py)

Uruchomienie modułowe: python3 -m scraper.main . Flagi: --limit N (test na N lotach), --dry-run (bez zapisu), --no-lock , -v (więcej logów). **Blokada pojedynczej instancji** (flock) zapobiega nakładaniu się przebiegów crona — drugi proces kończy się kodem 1.

13. Wtyczka WordPress — moduły

Plik główny (motocykle-poleasingowe.php)

Definiuje stałe: POLEA_VERSION (0.7.2), POLEA_PAGE_OPTION, POLEA_CACHE_TTL (5 minut). Rejestruje hooki aktywacji/dezaktywacji, shortcode (init) oraz styl frontu ładowany na żądanie.

Warstwa danych — Polea_DB (includes/class-db.php)

- łączy się przez mysqli_real_connect z **twardymi timeoutami** (connect 3 s, read 5 s) i wyłączonym LOCAL INFILE ; **nigdy nie przerywa** działania strony (zwraca null przy błędzie).
- Wszystkie zapytania to **prepared statements** (bind_param).
- query_list() — filtry + paginacja + sortowanie po terminie; get_one() , get_images() .
- first_images_map() — pobiera miniatury dla wielu lotów jednym zapytaniem (unikaj problemu N+1).
- distinct() — wartości filtrów z **cache (transient, 1 h)** ; status() — diagnostyka dla admina.

Bezpieczeństwo wejścia (includes/security.php)

- polea_sanitize_filters() — allowlista pól z \$_GET , sanityzacja, limity długości (64) i zakresów (rok 1900–2100, cena ≥ 0).
- polea_constrain_filters() — ogranicza filtry do **realnych wartości z bazy** i kubełkuje cenę (krok 500) — bramka przeciw zaśmiecaniu cache (storage DoS).
- polea_current_lot_id() — akceptuje wyłącznie ^[A-Za-z0-9]{1,32}\$.
- polea_flush_cache() — czyści transiency listy i wartości filtrów.

Front — shortcode [motocykle] (includes/shortcode.php)

- Renderuje **listę** (siatka + filtry + paginacja) lub **szczegóły** (gdy w URL jest ?motocykl=<lot_id>).
- Wynik **cache’owany** w transiencie na POLEA_CACHE_TTL (5 min), klucz zależny od filtrów.
- Całe wyjście **escapowane** (esc_html , esc_url , esc_attr); zdjęcia z referrerpolicy="no-referrer" .
- Gdy brak konfiguracji bazy → komunikat „Oferta motocykli będzie dostępna wkrótce.” (błędy nigdy nie trafiają do klienta).

Podstrona i motyw (includes/activation.php)

Przy aktywacji tworzy stronę „**Nasze motory**” (slug nasze-motory , treść [motocykle]) — idempotentnie — i wpina ją w menu: motyw blokowy (wp_navigation) lub klasyczny (lokalizacja menu). Styl dziedziczy kolory/fonty motywu (currentColor , color-mix).

Panel administratora (includes/admin.php)

Ekran **Ustawienia → Motocykle** : status połączenia z bazą, instrukcja wpisania stałych do wp-config.php , link do podstrony oraz przycisk **czyszczenia cache** (zabezpieczony nonce i uprawnieniem manage_options).

Deinstalacja (uninstall.php)

Usuwa podstronę, opcję i transiency wtyczki. **Nie dotyka** zewnętrznej bazy polea_* — należy ona do scrapera.

14. Interfejsy i „endpointy”

Brak własnych endpointów REST/AJAX

Wtyczka świadomie NIE rejestruje własnych tras REST ani akcji AJAX — zmniejsza to powierzchnię ataku. Cała interakcja odbywa się przez shortcode i parametry URL strony oraz standardowy ekran ustawień w panelu.

Publiczne parametry URL podstrony:

Parametr URL / atrybut	Znaczenie
?motocykl=<lot_id>	Widok szczegółów wybranego motocykla.
?polea_marka=...	Filtr: marka (walidowany do wartości z bazy).
?polea_paliwo=...	Filtr: paliwo.
?polea_rok=...	Filtr: rok produkcji.
?polea_cena_max=... / ?polea_cena_min=...	Filtr: cena (kubekowana co 500 PLN).
?polea_str=N	Numer strony listy (paginacja).
[motocykle ile="12"]	Atrybut shortcode: liczba kafelków na stronę (maks. 60).

Panel: **Ustawienia** → **Motocykle** (POST polea_flush z nonce polea_flush_cache).

15. Instalacja i wdrożenie — krok po kroku (A-Z)

15.1 Wymagania

- Serwer **VPS** z Pythonem 3 i dostępem sieciowym do poleasingowe.pl (dla scrapera).
- **MySQL 5.7+ / MariaDB 10.2+** (osobna baza polea).
- **WordPress 6.0+ / PHP 7.4+** z rozszerzeniem mysqli .

15.2 Utworzenie bazy i tabel

```
CREATE DATABASE polea CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
mysql -u root -p polea < db/schema.sql
```

15.3 Konto MySQL (zalecany rozdział uprawnień)

Zalecane są **dwa** konta: jedno dla scrapera (zapis) i jedno dla wtyczki (**tylko odczyt**). Nazwy/hosty i hasła ustala zespół wdrożeniowy:

```
-- Konto scrapera (zapis danych)
CREATE USER 'polea_scraper'@'localhost' IDENTIFIED BY 'HASLO_1';
GRANT SELECT, INSERT, UPDATE, DELETE ON polea.* TO 'polea_scraper'@'localhost';

-- Konto wtyczki WordPress (tylko odczyt)
CREATE USER 'polea_ro'@'localhost' IDENTIFIED BY 'HASLO_2';
GRANT SELECT ON polea.* TO 'polea_ro'@'localhost';
FLUSH PRIVILEGES;
```

15.4 Scraper na serwerze VPS

```
python3 -m venv .venv && . .venv/bin/activate
pip install -r scraper/requirements.txt

# poświadczenia bazy TYLK0 przez zmienne środowiskowe (nigdy w kodzie):
export POLEA_DB_HOST=127.0.0.1 POLEA_DB_NAME=polea \
    POLEA_DB_USER=polea_scraper POLEA_DB_PASSWORD='HASLO_1'
```

```
python3 -m scraper.main --dry-run --limit 3 # test bez zapisu
python3 -m scraper.main                    # pełny import
```

15.5 Harmonogram (cron) — import co pewien czas

Przykład: import co 3 godziny (dokładną częstotliwość potwierdza zespół). Blokada flock jest już wbudowana w scraper, więc nakładające się przebiegi nie zaszkodzą.

```
# crontab -e (na serwerze VPS)
0 */3 * * * cd /opt/polea && . .venv/bin/activate && \
  POLEA_DB_HOST=127.0.0.1 POLEA_DB_NAME=polea POLEA_DB_USER=polea_scraper \
  POLEA_DB_PASSWORD='HASLO_1' python3 -m scraper.main >> /var/log/polea.log 2>&1
```

Alternatywnie można użyć usługi **systemd + timer** (poświadczenia w EnvironmentFile z prawami 600). Wybór metody i ścieżki logów — do potwierdzenia przez zespół.

15.6 Wtyczka WordPress

1. Skopiuj katalog wp-plugin/motocykle-poleasingowe/ do wp-content/plugins/ i **aktywuj** wtyczkę (automatycznie tworzy się podstrona „Nasze motory”).
2. Dodaj poświadczenia bazy do wp-config.php (powyżej linii „That's all, stop editing”), używając konta **tylko do odczytu** :

```
define('POLEA_DB_HOST', '127.0.0.1');
define('POLEA_DB_NAME', 'polea');
define('POLEA_DB_USER', 'polea_ro');
define('POLEA_DB_PASSWORD', 'HASLO_2');
// opcjonalnie: define('POLEA_DB_PORT', 3306);
```

1. Sprawdź **Ustawienia → Motocykle** — status powinien pokazać „Połączono. Motocykli w bazie: N”.

15.7 Weryfikacja end-to-end

- Scraper: python3 -m scraper.main --dry-run --limit 3 kończy się bez błędów.
- Testy: python3 -m unittest scraper.tests.test_scraper -v (16 testów).
- WordPress: podstrona „Nasze motory” pokazuje kafelki; filtry i szczegóły działają.

16. Konfiguracja — zmienne środowiskowe scrapera

Zmienna	Domyślnie	Opis
POLEA_DB_HOST	127.0.0.1	Host bazy MySQL.
POLEA_DB_PORT	3306	Port bazy.
POLEA_DB_USER	polea	Użytkownik bazy (konto zapisu).
POLEA_DB_PASSWORD	(puste)	Hasło — wymagane, tylko przez env.
POLEA_DB_NAME	polea	Nazwa bazy.
POLEA_DB_CONNECT_TIMEOUT	10	Timeout połączenia [s].
POLEA_DB_READ_TIMEOUT	30	Timeout odczytu [s].
POLEA_DB_WRITE_TIMEOUT	30	Timeout zapisu [s].
POLEA_DB_SSL_CA	(brak)	Ścieżka do certyfikatu CA — włącza TLS do bazy.
POLEA_USER_AGENT	PoleasingoweImporter/0.1 (+kontakt)	Nagłówek User-Agent scrapera.

Zmienna	Domyślnie	Opis
POLEA_DELAY / POLEA_JITTER	2.0 / 1.0	Odstęp między żądaniami + losowy jitter [s].
POLEA_RETRIES	3	Liczba ponowień przy błędach sieci.
POLEA_TIMEOUT	30	Timeout pojedynczego żądania HTTP [s].
POLEA_MAX_PAGES	50	Bezpiecznik paginacji listy.
POLEA_MAX_REDIRECTS	3	Limit przekierowań (anty-SSRF).
POLEA_MAX_BYTES	8388608	Limit rozmiaru odpowiedzi (8 MB).
POLEA_MAX_TOTAL	60	Budżet czasu na jedną odpowiedź [s].
POLEA_LOCK	/tmp/polea_import.lock	Plik blokady pojedynczej instancji.
POLEA_TRUST_ENV	0	1 = ufaj proxy/.netrc ze środowiska (domyślnie nie).

Stałe w wp-config.php (wtyczka): POLEA_DB_HOST , POLEA_DB_NAME , POLEA_DB_USER , POLEA_DB_PASSWORD , opcjonalnie POLEA_DB_PORT .

17. Bezpieczeństwo (podsumowanie)

Pełny raport: docs/SECURITY-AUDIT.md (dwie iteracje utwardzania, wynik ~9,3/10). Powierzchnia ataku jest wąska — brak REST/AJAX, uploadu plików, eval / unserialize .

- **Wtyczka** : prepared statements, escapowanie wyjścia, nonce + uprawnienia w adminie, allowlisty i limity wejścia, mysqli zamiast wpdb (odporność na awarię bazy), LOCAL INFILE wyłączone, hotlink z referrerpolicy=no-referrer .
- **Scraper** : ochrona anty-SSRF (allowlista hostów, blokada IP prywatnych, walidacja przekierowań), limity rozmiaru i czasu odpowiedzi, flock , zaktualizowane zależności (CVE).

Zalecenia po stronie serwera (do wdrożenia przez administratora)

Konto MySQL tylko-SELECT dla wtyczki • TLS do bazy, jeśli baza jest zdalna (POLEA_DB_SSL_CA) • nagłówki bezpieczeństwa HTTP na serwerze WWW • monitoring importu i logów. Szczegóły w docs/SECURITY-AUDIT.md (pkt 5).

18. Utrzymanie i eksploatacja

- **Cache** : lista 5 min (transient), wartości filtrów 1 h. Ręczne czyszczenie: Ustawienia → Motocykle.
- **Aktualizacja wtyczki** : podbicie wersji unieważnia cache zasobów (CSS) po stronie przeglądarki.
- **Kopie zapasowe** : bazę polea_* można odtworzyć ponownym importem (źródło jest „prawdą”); bazę WordPressa backupuje się standardowo.
- **Monitoring** : warto obserwować logi scrapera oraz status w panelu (liczba motocykli, reconcile).
- **Retencja** : zakończone aukcje pozostają w bazie ze statusem zakonczona (polityka kasowania — do potwierdzenia).

Diagnostyka najczęstszych sytuacji:

Objaw	Prawdopodobna przyczyna	Działanie
„Oferta dostępna wkrótce”	Brak stałych POLEA_DB_* w wp-config.php	Uzupełnić konfigurację bazy.
„Motocykli w bazie: 0”	Scraper nie wykonał importu	Uruchomić import; sprawdzić cron i logi.

Objaw	Prawdopodobna przyczyna	Działanie
Brak tabeli polea_motocykle	Nie wgrano schematu	Wykonać db/schema.sql.
Zdjęcia się nie ładują	Hotlink zablokowany przez źródło	Zweryfikować dostępność zdjęć w źródle.
Stare dane mimo importu	Cache listy (5 min)	Wyczyścić cache w panelu.
Import się nie uruchamia	Aktywna blokada flock innego przebiegu	Sprawdzić, czy poprzedni proces nie wisi.

19. Wydajność

- Cache listy (5 min) i wartości filtrów (1 h) ograniczają liczbę zapytań do bazy.
- `first_images_map()` pobiera miniatury zbiorczo — brak problemu N+1 na liście.
- Indeksy bazy (marka , status , termin , vin , last_seen) przyspieszają filtrowanie i sortowanie.
- Scraper stosuje rate-limit (2 s + jitter) i bezpiecznik POLEA_MAX_PAGES , by nie obciążać źródła.

CZĘŚĆ III

Załączniki i informacje wymagające potwierdzenia

Otwarte decyzje wdrożeniowe, słownik pojęć, odnośniki

20. Wymaga potwierdzenia przez zespół projektowy

Poniższych elementów **nie da się jednoznacznie ustalić z samego kodu** — wymagają decyzji lub potwierdzenia zespołu:

- **Częstotliwość importu (cron)** — kod nie narzuca harmonogramu; założono „co kilka godzin”.
- **Środowisko bazy** — czy baza jest lokalna (127.0.0.1) czy zdalna; jeśli zdalna, czy wymagany TLS (POLEA_DB_SSL_CA) oraz jakie hosty/nazwy kont MySQL.
- **Rozdział kont MySQL** (zapis vs tylko-SELECT) — zalecany, ale to decyzja wdrożeniowa.
- **Docelowy User-Agent** scrapera i adres kontaktowy w nim zawarty.
- **Pola obecnie niewypełniane przez scraper** : `najniższa_cena_30d` , `tryb_licytacji` , `liczba_ofert` , `uwagi` — kolumny istnieją w schemacie, ale pipeline ich nie ustawia. Czy mają być uzupełniane?
- **Fallback zdjęć do Media Library** — wspomniany w decyzjach projektowych, nie zaimplementowany (obecnie wyłącznie hotlink).
- **Logi i monitoring** — docelowa lokalizacja logów scrapera, alerty, retencja logów.
- **Specyfikacja i dostęp do VPS** — hosting, wersja systemu, dostęp SSH, katalog wdrożenia.
- **Polityka retencji** zakończonych aukcji (czy i kiedy usuwać rekordy zakończona).
- **CI/CD oraz skrypty deploymentu** — w branchu plugin-2 **nie ma** dedykowanych plików CI ani skryptów deploymentu; jeśli są wymagane, należy je ustalić i utworzyć.

Uwaga o „skryptach deploymentu”

W treści zlecenia wspomniano o skryptach deploymentu. W obecnym stanie repozytorium (branch plugin-2) takie skrypty nie występują jako pliki — wdrożenie opisano ręcznie w rozdziale 15. Automatyzację wdrożenia (np. systemd/Ansible) należy potwierdzić z zespołem.

21. Słownik pojęć

Pojęcie	Znaczenie
<code>lot_id</code>	Stały identyfikator aukcji z adresu URL — klucz główny rekordu.
<code>relist</code> (wznowienie)	Ponowne wystawienie tego samego pojazdu (rozpoznanie po VIN) jako nowej aukcji.
<code>reconcile</code>	Zamknięcie ofert, które zniknęły ze źródła (status → zakończona).
<code>raw_hash</code>	Skrót MD5 rekordu do wykrywania zmian (nowy / zmieniony / bez zmian).
<code>hotlink</code>	Wyświetlanie zdjęcia bezpośrednio ze źródła, bez kopiowania na serwer klienta.
<code>upsert</code>	INSERT lub UPDATE — wstaw nowy albo zaktualizuj istniejący rekord.
<code>transient</code>	Wbudowany mechanizm pamięci podręcznej WordPressa z czasem ważności.
<code>shortcode</code>	Znacznik w treści strony (tu: [motocykle]) renderowany przez wtyczkę.
SSRF	Atak zmuszający serwer do żądań do zasobów wewnętrznych — tu blokowany allowlistą.
<code>flock</code>	Blokada pliku zapewniająca uruchomienie tylko jednej instancji importu naraz.

22. Odnośniki (dokumentacja w repozytorium)

- docs/ARCHITEKTURA.md — architektura działu/agencji/krytycy.
- docs/działy/*.md — szczegółowy opis każdego działu (1A, 1B, 2, 3, 4, 5, 6, 7, 8, 9, 10).
- docs/SECURITY-AUDIT.md — pełny raport audytu bezpieczeństwa i utwardzania.
- docs/klient/pdf/skad-pochodza-dane.pdf , jak-to-dziala.pdf — materiały poglądowe dla klienta.
- scraper/README.md , wp-plugin/motocykle-poleasingowe/README.md — skrócone instrukcje modułów.

Diagramy

Schematy architektury i przepływu danych dostarczane są jako osobne załączniki graficzne (uzupełniają rozdziały 8–14).